```
"""Uplink OS — a retro CRT terminal operating system for the evidence upload.

Replaces the old "hold [E] to upload" mechanic with a full interactive OS:
  1. BOOT     — CRT warm-up, BIOS POST, [illegible] OS boot screen ([illegible])
  2. LOGIN    — username pre-filled, password field ([illegible]), ENTER to log in
  3. DESKTOP  — 4 clickable icons: EVIDENCE, [illegible], SYSTEM, LOGS
  4. BROWSER  — ArchiveWeb browser with URL bar; navigate to upload site
  5. UPLOAD   — file picker (select 3 evidence files), progress bar with status
  6. DONE     — "UPLOAD COMPLETE", then transitions back to gameplay

The OS freezes enemies (handled by [illegible] when this is active).
Visual style: green phosphor CRT, scanlines, boot text, retro terminal font.
"""
import math
import random
import pygame

from config import WIDTH, HEIGHT, [illegible], audio
from state import state

# -- OS phase constants ----------------------------------------
PHASE_BOOT = "boot"
PHASE_LOGIN = "login"
PHASE_DESKTOP = "desktop"
PHASE_BROWSER = "browser"
PHASE_FILE_PICKER = "file_picker"
PHASE_UPLOAD = "upload"
PHASE_DONE = "done"
PHASE_INFO = "info"  # generic info window for [illegible] icons

# -- Colors (retro CRT green phosphor) -------------------------
[illegible]
```

```
                                    # Match known pages.
    if "Pixelate" in url:
        _browser_page = "Pixelate"
    elif "news" in url:
        _browser_page = "news"
    elif "weather" in url:
        _browser_page = "weather"
    elif "forum" in url:
        _browser_page = "forum"
    elif "upload" in url:
        _browser_page = "upload"
    else:
        _browser_page = "home"
    _browser_url = url if url else "scarlnet.net"


def _draw_chromatic_text(surface, font, text, pos, color, offset=2):
    """Render text with a subtle chromatic aberration (R left, B right).

    Only worth using for bright/highlight text — body text is too small
    for the effect to read. The offset is 2px by default.
    """
    # Red shadow (offset left).
    r_surf = font.render(text, True, (min(255, color[0]), max(0, color[1] - 60), max(0, color[2] - 60)))
    r_surf.set_alpha(100)
    surface.blit(r_surf, (pos[0] - offset, pos[1]))
    # Blue shadow (offset right).
    b_surf = font.render(text, True, (max(0, color[0] - 60), max(0, color[1] - 60), min(255, color[2] + 60)))
    b_surf.set_alpha(100)
    surface.blit(b_surf, (pos[0] + offset, pos[1]))
    # Main text.
    main = font.render(text, True, color)
    surface.blit(main, pos)


def _draw_radial_vignette(surface):
    """Draw a radial CRT vignette (darker corners, bright center).

    Replaces the old rectangular vignette (40 nested rectangles) with a
    single radial gradient surface. Sells the curved CRT glass.
    """
    # Create the vignette once per frame (cheap enough at 1280x720).
    vignette = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
    cx, cy = WIDTH // 2, HEIGHT // 2
    max_dist = math.hypot(cx, cy)
    # Draw concentric circles from outside in, increasing alpha.
    steps = 30
    for i in range(steps):
        t = i / steps  # 0 = center, 1 = corner
        radius = int(max_dist * (1 - t * 0.6))
        alpha = int(80 * t * t)  # quadratic falloff
        if alpha > 0:
            pygame.draw.circle(vignette, (0, 0, 0, alpha), (cx, cy), radius)
    surface.blit(vignette, (0, 0))


def _draw_custom_cursor(surface):
    """Draw a pixel-art CRT-green arrow cursor at the mouse position.

    The system cursor is hidden (set in start_upload_os). This draws a
    retro 1-bit arrow with a phosphor glow, matching the OS aesthetic.
    """
    mx, my = pygame.mouse.get_pos()
    # Arrow outline (pixel-art style — each pixel is 2x2 for visibility).
    p = 2  # pixel size
    # Arrow shape (1 = filled, 0 = empty).
    arrow = [
        [1, 0, 0, 0, 0, 0, 0],
        [1, 1, 0, 0, 0, 0, 0],
        [1, 1, 1, 0, 0, 0, 0],
        [1, 1, 1, 1, 0, 0, 0],
        [1, 1, 1, 1, 1, 0, 0],
        [1, 1, 1, 1, 1, 1, 0],
        [1, 1, 1, 1, 1, 1, 1],
        [1, 1, 0, 1, 1, 0, 0],
        [0, 0, 0, 1, 1, 0, 0],
        [0, 0, 0, 0, 1, 1, 0],
    ]
    # Draw glow first (offset copies at low alpha).
    glow_surf = pygame.Surface((7 * p + 4, 10 * p + 4), pygame.SRCALPHA)
    for gy in range(len(arrow)):
        for gx in range(len(arrow[gy])):
            if arrow[gy][gx]:
                pygame.draw.rect(glow_surf, (*CRT_BRIGHT, 40),
                                 (gx * p + 1, gy * p + 1, p + 2, p + 2))
    surface.blit(glow_surf, (mx - 1, my - 1))
    # Draw solid arrow.
    for gy in range(len(arrow)):
        for gx in range(len(arrow[gy])):
            if arrow[gy][gx]:
                pygame.draw.rect(surface, CRT_BRIGHT,
                                 (mx + gx * p, my + gy * p, p, p))


def handle_upload_os_event(event):
    """Handle a pygame event (keyboard/mouse) for the upload OS.

    Returns True if the event was consumed by the OS, False otherwise.
    """
    global _os_phase, _os_timer, _login_password, _login_error
    global _browser_url, _browser_page, _file_picker_selected
    global _upload_progress, _upload_status_idx, _info_content
    global _login_attempts, _login_lockout_timer
    global _url_editing, _upload_intrusion_active, _menu_timer
    global _info_sub, _start_menu_open, _start_submenu
    global _run_dialog_open, _run_dialog_text
    global _last_click_time, _last_clicked_file

    if not _os_active:
        return False

    # -- BOOT: ENTER to skip --
    if _os_phase == PHASE_BOOT:
        if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
            _os_phase = PHASE_LOGIN
            _os_timer = 0
            return True
        return True  # consume all events during boot

    # -- LOGIN: Type password, ENTER to submit (with lockout) --
    if _os_phase == PHASE_LOGIN:
        # If locked out, reject all input except ESC (which does nothing but consume).
        if _login_lockout_timer > 0:
            return True
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN or event.key == pygame.K_KP_ENTER:
                if _login_password.lower() in ("admin", "admin/admin", ""):
                    _os_phase = PHASE_DESKTOP
                    _os_timer = 0
                    _login_attempts = 0  # reset on success
                    audio.play("menu_click", volume=0.4)
                else:
                    _login_attempts += 1
                    if _login_attempts >= 3:
                        _login_error = "ACCOUNT LOCKED - 10s"
                        _login_lockout_timer = 600  # 10 seconds at 60fps
                        audio.play("empty_click", volume=0.5)
                    else:
                        _login_error = f"ACCESS DENIED - Invalid credentials ({_login_attempts}/3)"
                        audio.play("empty_click", volume=0.5)
                return True
            elif event.key == pygame.K_BACKSPACE:
                _login_password = _login_password[:-1]
                _login_error = ""
                return True
            elif event.unicode and event.unicode.isprintable() and len(_login_password) < 20:
                _login_password += event.unicode
                _login_error = ""
                audio.play("type_tick", volume=0.15)
                return True
        return True

    # -- UPLOAD: intrusion popup dismissal --
    if _os_phase == PHASE_UPLOAD and _upload_intrusion_active:
        if event.type == pygame.KEYDOWN or (
            event.type == pygame.MOUSEBUTTONDOWN and event.button == 1
        ):
            _upload_intrusion_active = False
            audio.play("menu_click", volume=0.4)
        return True  # consume all events while the popup is up

    # -- Run dialog (modal over desktop) --
    if _os_phase == PHASE_DESKTOP and _run_dialog_open:
        # Mouse clicks on OK / Cancel buttons.
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            mx, my = event.pos
            # Button positions (must match _draw_run_dialog).
            dw, dh = 400, 160
            dx = WIDTH // 2 - dw // 2
            dy = HEIGHT // 2 - dh // 2
            body_right = dx + 4 + dw - 8
            body_bottom = dy + 30 + dh - 34
            ok_rect = pygame.Rect(body_right - 180, body_bottom - 34, 80, 24)
            cancel_rect = pygame.Rect(body_right - 90, body_bottom - 34, 80, 24)
            if ok_rect.collidepoint(mx, my):
                _run_dialog_open = False
                if _run_dialog_text.strip():
                    _os_phase = PHASE_BROWSER
                    _browser_url = _run_dialog_text.strip()
                    _navigate_to_url(_run_dialog_text.strip())
                    _run_dialog_text = ""
                    audio.play("menu_click", volume=0.4)
                else:
                    audio.play("empty_click", volume=0.3)
                return True
            if cancel_rect.collidepoint(mx, my):
                _run_dialog_open = False
                _run_dialog_text = ""
                audio.play("empty_click", volume=0.3)
                return True
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN or event.key == pygame.K_KP_ENTER:
                _run_dialog_open = False
                if _run_dialog_text.strip():
                    _os_phase = PHASE_BROWSER
                    _browser_url = _run_dialog_text.strip()
                    _navigate_to_url(_run_dialog_text.strip())
                    _run_dialog_text = ""
                    audio.play("menu_click", volume=0.4)
                else:
                    audio.play("empty_click", volume=0.3)
                return True
            elif event.key == pygame.K_ESCAPE:
                _run_dialog_open = False
                _run_dialog_text = ""
                audio.play("empty_click", volume=0.3)
                return True
            elif event.key == pygame.K_BACKSPACE:
                _run_dialog_text = _run_dialog_text[:-1]
                audio.play("type_tick", volume=0.1)
                return True
            elif event.unicode and event.unicode.isprintable() and len(_run_dialog_text) < 40:
                _run_dialog_text += event.unicode
                audio.play("type_tick", volume=0.1)
                return True
        return True  # consume all events while the Run dialog is open

    # -- Start menu (overlays on desktop) --
    if _os_phase == PHASE_DESKTOP and _start_menu_open:
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            mx, my = event.pos
            # Start menu position (must match _draw_start_menu).
            sm_x = 0
            sm_y = HEIGHT - 36 - 280
            sm_w = 200
            sm_h = 280
            # Click outside the menu closes it.
            if not (sm_x <= mx <= sm_x + sm_w and sm_y <= my <= sm_y + sm_h):
                _start_menu_open = False
                _start_submenu = None
                audio.play("empty_click", volume=0.3)
                return True
            # Menu items.
            items = [
                ("programs", 0), ("documents", 1), ("settings", 2),
                ("find", 3), ("help", 4), ("run", 5), ("shutdown", 6),
            ]
            for item_id, idx in items:
                iy = sm_y + 8 + idx * 36
                # Sidebar width = 30 (matches _draw_start_menu). Items start
                # after the sidebar so the "VOS" banner isn't clickable.
                sidebar_w = 30
                item_rect = pygame.Rect(sm_x + sidebar_w + 4, iy, sm_w - sidebar_w - 8, 32)
                if item_rect.collidepoint(mx, my):
                    if item_id == "programs":
                        # Toggle submenu.
                        _start_submenu = "programs" if _start_submenu != "programs" else None
                        audio.play("menu_hover", volume=0.2)
                    elif item_id == "documents":
                        _start_submenu = "documents" if _start_submenu != "documents" else None
                        audio.play("menu_hover", volume=0.2)
                    elif item_id == "settings":
                        _start_menu_open = False
                        _start_submenu = None
                        _os_phase = PHASE_INFO
                        _info_content = "system"
                        _info_sub = None
                        audio.play("menu_click", volume=0.4)
                    elif item_id == "run":
                        _start_menu_open = False
                        _start_submenu = None
                        _run_dialog_open = True
                        _run_dialog_text = ""
                        audio.play("menu_click", volume=0.4)
                    elif item_id == "shutdown":
                        _start_menu_open = False
                        _start_submenu = None
                        # Shut down the OS — returns to gameplay.
                        deactivate_upload_os()
                        audio.play("menu_click", volume=0.4)
                    else:
                        # Find / Help — no action, just close.
                        _start_menu_open = False
                        _start_submenu = None
                        audio.play("empty_click", volume=0.3)
                    return True
            # Submenu clicks (Programs / Documents).
            if _start_submenu == "programs":
                sub_items = ["evidence", "scarlnet", "system", "logs"]
                sub_x = sm_x + sm_w
                for i, sid in enumerate(sub_items):
                    sy = sm_y + 8 + i * 36
                    sub_rect = pygame.Rect(sub_x + 4, sy, 160, 32)
                    if sub_rect.collidepoint(mx, my):
                        _start_menu_open = False
                        _start_submenu = None
                        if sid == "scarlnet":
                            _os_phase = PHASE_BROWSER
                            _browser_page = "home"
                        else:
                            _os_phase = PHASE_INFO
                            _info_content = sid
                            _info_sub = None
                        audio.play("menu_click", volume=0.4)
                        return True
            elif _start_submenu == "documents":
                doc_ids = ["dossier", "swift_chats", "bug_packet"]
                sub_x = sm_x + sm_w
                for i, did in enumerate(doc_ids):
                    sy = sm_y + 8 + i * 36
                    sub_rect = pygame.Rect(sub_x + 4, sy, 160, 32)
                    if sub_rect.collidepoint(mx, my):
                        _start_menu_open = False
                        _start_submenu = None
                        _os_phase = PHASE_INFO
                        _info_content = "evidence"
                        _info_sub = did  # open the document viewer directly
                        audio.play("menu_click", volume=0.4)
                        return True
            return True
        if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
            _start_menu_open = False
            _start_submenu = None
            return True
        # Mouse motion updates the hovered submenu (for cascade feel).
        if event.type == pygame.MOUSEMOTION:
            mx, my = event.pos
            sm_x = 0
            sm_y = HEIGHT - 36 - 280
            # If hovering Programs/Documents, auto-open that submenu.
            for item_id, idx in [("programs", 0), ("documents", 1)]:
                iy = sm_y + 8 + idx * 36
                sidebar_w = 30  # match _draw_start_menu
                item_rect = pygame.Rect(sm_x + sidebar_w + 4, iy, 200 - sidebar_w - 8, 32)
                if item_rect.collidepoint(mx, my):
                    if _start_submenu != item_id:
                        _start_submenu = item_id
                    return True
        return True  # consume all events while the Start menu is open

    # -- DESKTOP: Click icons + Start button --
    if _os_phase == PHASE_DESKTOP:
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            mx, my = event.pos
            # Start button (bottom-left, in the taskbar).
            start_btn_rect = pygame.Rect(0, HEIGHT - 36 + 4, 80, 28)
            if start_btn_rect.collidepoint(mx, my):
                _start_menu_open = not _start_menu_open
                _start_submenu = None
                audio.play("menu_click", volume=0.4)
                return True
            # Desktop icons.
            icon_rects = _get_desktop_icon_click_rects()
            for i, icon in enumerate(DESKTOP_ICONS):
                rect = icon_rects[i]
                if rect.collidepoint(mx, my):
                    if icon["id"] == "scarlnet":
                        _os_phase = PHASE_BROWSER
                        _os_timer = 0
                        _browser_page = "home"
                        audio.play("menu_click", volume=0.4)
                    elif icon["id"] == "evidence":
                        _os_phase = PHASE_INFO
                        _info_content = "evidence"
                        _info_sub = None  # start at file manager
                        audio.play("menu_click", volume=0.4)
                    elif icon["id"] == "system":
                        _os_phase = PHASE_INFO
                        _info_content = "system"
                        _info_sub = None  # start at control panel
                        audio.play("menu_click", volume=0.4)
                    elif icon["id"] == "logs":
                        _os_phase = PHASE_INFO
                        _info_content = "logs"
                        _info_sub = None
                        audio.play("menu_click", volume=0.4)
                    return True
        return True

    # -- INFO: file manager / control panel / logs / document viewer --
    if _os_phase == PHASE_INFO:
        # Window position (must match _draw_info).
        win_x = WIDTH // 2 - 380
        win_y = 100
        win_w, win_h = 760, 460  # bigger window for the richer content
        win_rect = pygame.Rect(win_x, win_y, win_w, win_h)
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            mx, my = event.pos
            # Click outside the window closes it.
            if not win_rect.collidepoint(mx, my):
                _os_phase = PHASE_DESKTOP
                _info_content = None
                _info_sub = None
                audio.play("menu_click", volume=0.3)
                return True
            # Close [X] button — top-right of title bar.
            close_rect = pygame.Rect(win_x + win_w - 30, win_y + 4, 24, 20)
            if close_rect.collidepoint(mx, my):
                _os_phase = PHASE_DESKTOP
                _info_content = None
                _info_sub = None
                audio.play("menu_click", volume=0.3)
                return True
            # Back button (when in a sub-view like a document or control panel tab).
            if _info_sub is not None:
                back_rect = pygame.Rect(win_x + 10, win_y + win_h - 36, 100, 26)
                if back_rect.collidepoint(mx, my):
                    _info_sub = None
                    audio.play("menu_click", volume=0.3)
                    return True
            # -- Evidence file manager: file double-click --
            if _info_content == "evidence" and _info_sub is None:
                # File list area (right pane). The x/y offset matches the
                # column headers drawn above the file rows in
                # _draw_evidence_file_manager (HEADER/ROWSTART).
                list_x = win_x + 160
                list_y = win_y + 80
                doc_ids = ["dossier", "swift_chats", "bug_packet"]
                for i, did in enumerate(doc_ids):
                    ry = list_y + 20 + i * 28  # +20 to skip the column header row
                    row_rect = pygame.Rect(list_x, ry, win_w - 170, 26)
                    if row_rect.collidepoint(mx, my):
                        now = pygame.time.get_ticks()
                        # Double-click detection: same file, within 500ms.
                        if (_last_clicked_file == i and now - _last_click_time < 500):
                            _info_sub = did  # open the document viewer
                            audio.play("menu_click", volume=0.4)
                        else:
                            _last_clicked_file = i
                            _last_click_time = now
                            audio.play("type_tick", volume=0.15)
                        return True
            # -- System control panel: icon clicks --
            if _info_content == "system" and _info_sub is None:
                # Grid of 6 icons (3 columns x 2 rows, left to right, row by row).
                # Anchored to body_rect (window body, below the title bar) —
                # must match _draw_control_panel_grid exactly.
                grid_x = win_x + 4 + 40  # body_rect.x + 40 (body_rect.x = win_x + 4)
                grid_y = win_y + 30 + 50  # body_rect.y + 50 (body_rect.y = win_y + 30)
                icon_w, icon_h = 80, 80
                gap_x, gap_y = 140, 100
                for i, item in enumerate(CONTROL_PANEL_ITEMS):
                    col = i % 3
                    row = i // 3
                    ix = grid_x + col * (icon_w + gap_x)
                    iy = grid_y + row * (icon_h + gap_y)
                    icon_rect = pygame.Rect(ix, iy, icon_w, icon_h)
                    if icon_rect.collidepoint(mx, my):
                        _info_sub = item["id"]
                        audio.play("menu_click", volume=0.4)
                        return True
        if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
            if _info_sub is not None:
                _info_sub = None
                audio.play("menu_click", volume=0.3)
            else:
                _os_phase = PHASE_DESKTOP
                _info_content = None
                _info_sub = None
                audio.play("menu_click", volume=0.3)
            return True
        return True

    # -- BROWSER: Click links/buttons + URL bar editing --
    if _os_phase == PHASE_BROWSER:
        # -- URL bar editing (keyboard input when _url_editing is True) --
        if _url_editing:
            if event.type == pygame.KEYDOWN:
                if event.key == pygame.K_RETURN or event.key == pygame.K_KP_ENTER:
                    # Navigate to the typed URL.
                    _url_editing = False
                    _navigate_to_url(_browser_url)
                    audio.play("menu_click", volume=0.4)
                    return True
                elif event.key == pygame.K_ESCAPE:
                    _url_editing = False
                    _browser_url = BROWSER_PAGES.get(_browser_page, {}).get("url", "scarlnet.net")
                    audio.play("empty_click", volume=0.3)
                    return True
                elif event.key == pygame.K_BACKSPACE:
                    _browser_url = _browser_url[:-1]
                    audio.play("type_tick", volume=0.1)
                    return True
                elif event.unicode and event.unicode.isprintable() and len(_browser_url) < 60:
                    _browser_url += event.unicode
                    audio.play("type_tick", volume=0.1)
                    return True
            return True  # consume all events while editing

        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            mx, my = event.pos
            win_x = WIDTH // 2 - 360
            win_y = 40
            body_x = win_x + 4
            body_y = win_y + 30
            tb_y = body_y + 8
            page_y = tb_y + 32
            # -- URL bar click = start editing --
            url_rect = pygame.Rect(body_x + 170, tb_y, 700 - 180, 24)
            if url_rect.collidepoint(mx, my):
                _url_editing = True
                audio.play("type_tick", volume=0.15)
                return True
            # -- Toolbar buttons: Back / Forward / Home --
            back_rect = pygame.Rect(body_x + 10, tb_y, 50, 24)
            fwd_rect = pygame.Rect(body_x + 64, tb_y, 50, 24)
            home_rect = pygame.Rect(body_x + 118, tb_y, 50, 24)
            if back_rect.collidepoint(mx, my):
                if _browser_page != "home":
                    _browser_page = "home"
                    _browser_url = "scarlnet.net"
                    audio.play("menu_click", volume=0.4)
                else:
                    audio.play("empty_click", volume=0.3)
                return True
            if fwd_rect.collidepoint(mx, my):
                audio.play("empty_click", volume=0.3)
                return True
            if home_rect.collidepoint(mx, my):
                if _browser_page != "home":
                    _browser_page = "home"
                    _browser_url = "scarlnet.net"
                    audio.play("menu_click", volume=0.4)
                else:
                    audio.play("empty_click", volume=0.3)
                return True
            # -- Page-specific action buttons + site links --
            if _browser_page == "home":
                # "Visit Upload Portal" button.
                page_cx = body_x + 8 + (700 - 8 - 16) // 2
                link_rect = pygame.Rect(page_cx - 140, page_y + 240, 280, 36)
                if link_rect.collidepoint(mx, my):
                    _browser_page = "upload"
                    _browser_url = "scarlnet.net/upload"
                    audio.play("menu_click", volume=0.4)
                    return True
                # Site links (clickable text lines for news/weather/forum).
                sites = [
                    ("news", "scarlnet.net/news", page_y + 140),
                    ("weather", "scarlnet.net/weather", page_y + 160),
                    ("upload", "scarlnet.net/upload", page_y + 180),
                    ("forum", "scarlnet.net/forum", page_y + 200),
                ]
                for page_id, url, sy in sites:
                    site_rect = pygame.Rect(body_x + 30, sy - 2, 700 - 70, 18)
                    if site_rect.collidepoint(mx, my):
                        _browser_page = page_id
                        _browser_url = url
                        audio.play("menu_click", volume=0.4)
                        return True
            elif _browser_page == "upload":
                page_cx = body_x + 8 + (700 - 8 - 16) // 2
                attach_rect = pygame.Rect(page_cx - 80, page_y + 180, 160, 36)
                if attach_rect.collidepoint(mx, my):
                    _os_phase = PHASE_FILE_PICKER
                    _os_timer = 0
                    audio.play("menu_click", volume=0.4)
                    return True
        if event.type == pygame.KEYDOWN and event.key == pygame.K_ESCAPE:
            _os_phase = PHASE_DESKTOP
            audio.play("menu_click", volume=0.3)
            return True
        return True

    # -- FILE PICKER: select file, then UPLOAD --
    if _os_phase == PHASE_FILE_PICKER:
        if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            mx, my = event.pos
            # File picker window position (must match _draw_file_picker).
            win_x = WIDTH // 2 - 300
            win_y = 80
            body_x = win_x + 4
            body_y = win_y + 30
            body_w = 600 - 8
            # Check file row clicks.
            for i in range(len(EVIDENCE_FILES)):
                row_y = body_y + 40 + i * 28
                row_rect = pygame.Rect(body_x + 10, row_y, body_w - 20, 26)
```